

ĐỀ THI MẪU — AICB · VinAI in Action

Thời gian làm bài: 120 phút

Tổng điểm: 100 điểm

Hình thức: Tự luận + Code chạy (được dùng máy tính cho Module B)

Cấu trúc: 4 Module — A (20đ) · B (25đ) · C (35đ) · D (20đ)

Đề bám sát nội dung các buổi học: Day 5 — Thiết kế sản phẩm AI cho sự không chắc chắn, Day 7 — Data Foundations (Embedding & Vector Store), Day 8 — RAG Pipeline, Day 11 — Guardrails & AI Safety, Day 19 — GraphRAG & Knowledge Graphs, Day 21 — Fine-tuning LLMs (LoRA/QLoRA).

Phần ĐÁP ÁN & THANG ĐIỂM nằm ở cuối đề.

MODULE A — Toán học & Tư duy Định lượng (20 điểm)

Tính tay, trình bày từng bước. Được làm tròn 2 chữ số thập phân khi cần.

Câu A1 — Ma trận nghịch đảo (5đ)

Cho ma trận:

$$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$$

- a) Tính định thức $\det(A)$. (1đ)
b) Tính ma trận nghịch đảo A^{-1} . (3đ)
c) Nếu $\det(A) = 0$ thì điều gì xảy ra với A^{-1} ? Giải thích ngắn gọn. (1đ)

Câu A2 — Tích vô hướng & Tích có hướng (5đ)

Cho hai vector trong không gian 3 chiều:

$$\vec{u} = (1, 2, 3), \quad \vec{v} = (4, 5, 6)$$

- a) Tính tích vô hướng (dot product) $\vec{u} \cdot \vec{v}$. (2đ)
b) Tính tích có hướng (cross product) $\vec{u} \times \vec{v}$. (3đ)

Câu A3 — Vector Norms: L1 vs L2 (5đ)

Cho vector sai số (residual) $\vec{e} = (3, -4)$.

- a) Tính **L1 Norm** (Manhattan distance) của $\vec{e}$. (1.5đ)
b) Tính **L2 Norm** (Euclidean distance) của $\vec{e}$. (1.5đ)
c) Trong bài toán regularization của AI, hãy nêu **một khác biệt cốt lõi** về tác dụng giữa phạt theo L1 (Lasso) và phạt theo L2 (Ridge). (2đ)

Câu A4 — Cosine Similarity (5đ) (liên hệ slide Embedding — Day 7)

Trong hệ thống tìm kiếm vector, hai câu được embedding thành hai vector:

$$\vec{a} = (1, 0, 1), \quad \vec{b} = (1, 1, 0)$$

a) Tính **cosine similarity** $\cos(\theta) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$. (3đ)

b) Theo slide, **Cosine** và **Euclidean** khác nhau ở chỗ nào, và vì sao **Cosine** là lựa chọn mặc định cho **text embedding**? (2đ)

MODULE B — Kỹ thuật Lập trình & Xử lý dữ liệu (25 điểm)

2 câu code chạy + 2 câu ngắn. Có thể dùng Python thuần hoặc NumPy theo yêu cầu từng câu.

Câu B1 — Code 1: Xử lý văn bản cho NLP (8đ)

Cho chuỗi văn bản "bắn" lấy từ log nội bộ:

```
python
```

```
raw = "  Khách HÀNG rất hài lòng!! Sản phẩm tốt, sản phẩm bền... Giao hàng NHAN
```

Viết hàm `clean_and_count(text)` thực hiện đúng pipeline tiền xử lý:

1. Chuẩn hóa khoảng trắng thừa (gộp nhiều space về 1, bỏ space đầu/cuối).
2. Chuyển toàn bộ về **chữ thường**.
3. Loại bỏ ký tự đặc biệt / dấu câu (chỉ giữ chữ và khoảng trắng).
4. Tách từ (tokenize).
5. Trả về `dict` đếm **tần suất xuất hiện của mỗi từ**, và in ra từ xuất hiện **nhiều nhất**.

Yêu cầu code chạy được với biến `raw` ở trên.

Câu hỏi phụ (1 trong 8đ): Nếu trong chuỗi có PII như số điện thoại `0901234567`, theo slide Day 8 ta phải làm gì với nó **trước khi** đẩy lên Vector DB trên cloud?

Câu B2 — Code 2: Viết lại công thức toán bằng Python (7đ)

Cho công thức **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- a) Viết hàm `mse_pure(y_true, y_pred)` bằng **Python thuần** (không dùng thư viện), nhận 2 list số thực. (3đ)
- b) Viết lại `mse_numpy(y_true, y_pred)` bằng **NumPy** (vector hóa, không dùng vòng lặp). (2đ)
- c) Viết thêm hàm kích hoạt **ReLU** dạng NumPy: $\text{ReLU}(x) = \max(0, x)$ áp dụng lên cả một mảng. (2đ)

Câu B3 — NumPy Broadcasting (4đ)

Cho đoạn code:

```
python

import numpy as np
A = np.array([[1], [2], [3]])      # shape (3, 1)
B = np.array([10, 20, 30, 40])    # shape (4,)
C = A + B
```

- a) `C.shape` là gì? Giải thích cơ chế broadcasting đã diễn ra. (2đ)
- b) Cho `x = np.array([1, 2, 3])` (shape `(3,)`) và `y = np.array([[1], [2], [3]])` (shape `(3, 1)`). `(x + y).shape` là gì? (2đ)

Câu B4 — Tính F1-Score bằng Python (6đ)

Một mô hình phân loại nhị phân cho ra: **TP = 8, FP = 5, FN = 2**.

Viết hàm `f1_score(tp, fp, fn)` tính **Precision**, **Recall** và **F1-Score** theo định nghĩa:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot P \cdot R}{P + R}$$

Hàm trả về cả 3 giá trị (làm tròn 3 chữ số). Xử lý trường hợp mẫu số bằng 0 để tránh lỗi chia cho 0.

MODULE C — Kiến thức AI & Tư duy sản phẩm AI (35 điểm)

Cấu trúc: 2 câu Thiết kế (C1, C2) + 1 câu Đánh giá (C3). Trình bày có cấu trúc, gạch đầu dòng được khuyến khích.

Câu C1 — Thiết kế hệ thống AI (13đ)

Business case: Một Sở Giáo dục muốn xây **hệ thống chấm bài thi tự động** cho giáo viên. Bài thi gồm 2 phần:

- Phần trắc nghiệm (MCQ) — học sinh tô đáp án trên phiếu giấy.
- Phần tự luận — học sinh viết tay/đánh máy, chấm dựa trên **đáp án mẫu (rubric)** do giáo viên cung cấp.

Hãy trình bày bản thiết kế hệ thống, bắt buộc nêu rõ:

- a) **Lựa chọn công nghệ** cho từng phần (gợi ý: OCR, CV, LLM/NLP) — công nghệ nào dùng để làm gì và **tại sao**. (4đ)
- b) **Pipeline phát triển sản phẩm** đầy đủ theo các bước: Thu thập dữ liệu → Tiền xử lý → Trích xuất đặc trưng → Lựa chọn mô hình → Đánh giá hiệu năng. (4đ)
- c) Với phần **chăm tự luận dựa trên đáp án mẫu**, hãy đề xuất kiến trúc **RAG** (hoặc **GraphRAG**) và giải thích vì sao RAG phù hợp hơn so với chỉ dùng một LLM trả lời trực tiếp. Nêu rõ 3 pha của RAG. (5đ)

Câu C2 — Triển khai & Tối ưu LLM (GenAI) (12đ)

Tình huống: Công ty cần một trợ lý LLM nội bộ **chuyên trả lời về quy trình pháp chế** bằng tiếng Việt, dữ liệu nhạy cảm, không được gửi ra ngoài.

- a) Theo **cây quyết định** đã học (Prompt → RAG → Fine-tune), khi nào nên chọn Prompting, khi nào RAG, khi nào Fine-tune? Với tình huống trên bạn chọn hướng nào và vì sao? (3đ)
- b) Giả sử cần **fine-tune** để model nói đúng văn phong pháp chế. Giải thích cơ chế **LoRA**: vì sao chỉ cần train rất ít tham số? Nêu vai trò của ma trận A, B và công thức $\Delta W = B \cdot A$. **QLoRA** khác LoRA ở điểm gì? (4đ)
- c) **Secure Deployment**: liệt kê các biện pháp **guardrails** để triển khai LLM an toàn, bao gồm: chống **Prompt Injection**, bảo vệ PII / **data leakage**, và kiểm soát **output độc hại**. Phân biệt **input guardrails** và **output guardrails**. (5đ)

Câu C3 — Đánh giá AI / AI Evaluation (10đ)

Phần 1 — Chọn metric (6đ). Một AI lọc nội dung xấu cho **ứng dụng dành cho trẻ em**: có 100 video, trong đó **10 video thực sự xấu**. Kết quả AI:

- AI đánh dấu "xấu" tổng cộng 13 video, trong đó **8 đúng là xấu (TP)**, **5 video tốt bị gán nhầm (FP)**.
- 2 video xấu bị bỏ lọt (FN)**, lọt vào tay trẻ em.
- 85 video tốt được cho qua đúng (TN).

- a) Tính **Precision** và **Recall**. (3đ)
- b) Trong bối cảnh **app trẻ em**, nên ưu tiên tối đa hóa **Precision** hay **Recall**? Giải thích theo "rule of thumb": loại sai nào (bỏ lọt FN hay báo nhầm FP) gây hậu quả nặng hơn? So sánh với bối cảnh **duyet vay ngân hàng** để thấy đáp án phụ thuộc context. (3đ)

Phần 2 — Đánh giá hệ thống RAG (4đ). Bạn build một chatbot RAG nhưng nó vẫn trả lời sai. Dùng bộ ba **RAGAS Triad**:

- a) Nêu tên và định nghĩa ngắn 3 trục: **Context Recall**, **Faithfulness**, **Answer Relevance**. (2đ)
- b) Nếu hệ thống có **Faithfulness thấp nhưng Context Recall cao** thì lỗi nằm ở đâu (Retrieval hay Generation) và cách khắc phục? Ngược lại, **Context Recall thấp** thì sửa khâu nào? (2đ)

MODULE D — Tư duy Logic, Đạo đức & Hành vi (20 điểm)

Phần này thiên về đọc kỹ tình huống và lập luận. Không có công thức.

Câu D1 — Tư duy Logic (6đ)

Một hệ thống agent có quy tắc định tuyến (routing) như sau:

- Nếu **confidence** ≥ 0.9 $\rightarrow$ agent tự động trả lời.
- Nếu **0.7** $\leq$ **confidence** < 0.9 $\rightarrow$ đưa vào hàng đợi cho người review (priority thường).
- Nếu **confidence** < 0.7 $\rightarrow$ chuyển ngay cho con người (priority cao).
- **Ngoại lệ:** mọi hành động thuộc nhóm ["gửi email", "xóa dữ liệu", "chuyển tiền"] $\rightarrow$ **luôn** chuyển cho con người, **bất kể** confidence.

Với mỗi yêu cầu sau, hãy xác định agent sẽ làm gì:

1. Tra cứu thông tin, (confidence = 0.95), action = ("trả lời"). (1.5đ)
2. Soạn và gửi email cho khách, (confidence = 0.98). (1.5đ)
3. Tóm tắt văn bản, (confidence = 0.75), action = ("trả lời"). (1.5đ)
4. Xóa dữ liệu một bản ghi, (confidence = 0.6). (1.5đ)

Câu D2 — An toàn AI & Đạo đức (8đ)

a) **Case thực tế (Air Canada 2024):** một chatbot tự "bịa" ra chính sách hoàn tiền không có thật, tòa phán hãng phải bồi thường. Bài học rút ra cho người thiết kế sản phẩm AI là gì? Liên hệ với khái niệm **Hallucination** và **Grounding**. (3đ)

b) Nêu **3 nguyên tắc** trong các framework quản trị AI (gợi ý: Fairness, Transparency, Accountability, Privacy, Safety) và giải thích ngắn. Kể tên ít nhất **2 framework / quy định** quản trị AI đã học. (3đ)

c) Theo mô hình **HITL (Human-in-the-loop)**, liệt kê 3 tình huống bắt buộc phải có con người tham gia quyết định (trigger). Vì sao nói "**HITL là feature, không phải là thừa nhận AI yếu**"? (2đ)

Câu D3 — Định kiến (Bias) & Bảo mật (6đ)

Tình huống: Một công ty train mô hình sàng lọc CV ứng viên. Dữ liệu lịch sử 10 năm cho thấy 95% người được tuyển ở vị trí kỹ thuật là nam giới. Mô hình mới học theo dữ liệu này và bắt đầu **tự động loại CV của ứng viên nữ**.

a) Đây là loại lỗi gì? Nguồn gốc của bias nằm ở đâu? (2đ)

b) Đề xuất **2 biện pháp** giảm thiểu bias cho hệ thống này. (2đ)

c) Trước khi đưa dữ liệu CV (chứa tên, SĐT, địa chỉ) vào hệ thống/đám mây, cần xử lý gì để tuân thủ bảo mật (PII)? (2đ)

✓ ĐÁP ÁN & THANG ĐIỂM

Đáp án bám sát công thức và khái niệm trong slide nội bộ. Với câu tự luận, đây là khung chấm — thí sinh nêu đủ ý chính là được điểm.

A1 — Ma trận nghịch đảo

- a) $\det(A) = 4 \cdot 6 - 7 \cdot 2 = 24 - 14 = \mathbf{10}$.
- b) Công thức cho ma trận 2×2 : $A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

$$A^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}$$

- c) Nếu $\det(A) = 0 \rightarrow$ ma trận **suy biến (singular)**, **không tồn tại** ma trận nghịch đảo (chia cho 0).

A2 — Dot & Cross product

- a) $\vec{u} \cdot \vec{v} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = \mathbf{32}$.
- b) $\vec{u} \times \vec{v} = (u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1)$
 $= (2 \cdot 6 - 3 \cdot 5, 3 \cdot 4 - 1 \cdot 6, 1 \cdot 5 - 2 \cdot 4) = (12 - 15, 12 - 6, 5 - 8) = (\mathbf{-3, 6, -3})$

A3 — Norms

- a) $L1 = |3| + |-4| = \mathbf{7}$.
- b) $L2 = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = \sqrt{25} = \mathbf{5}$.
- c) $L1$ (Lasso) tạo nghiệm **thưa (sparse)** — đẩy nhiều trọng số về đúng 0 $\rightarrow$ giúp **chọn đặc trưng (feature selection)**. $L2$ (Ridge) **co nhỏ đều** các trọng số, hiếm khi về 0 $\rightarrow$ chống overfitting nhưng giữ lại mọi đặc trưng.

A4 — Cosine similarity

- a) $\vec{a} \cdot \vec{b} = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 1; \|\vec{a}\| = \sqrt{2}, \|\vec{b}\| = \sqrt{2}$.

$$\cos \theta = \frac{1}{\sqrt{2} \cdot \sqrt{2}} = \frac{1}{2} = \mathbf{0.5} \quad (\theta = 60^\circ)$$

- b) **Cosine** đo góc giữa hai vector (giá trị $-1 \rightarrow 1$, càng gần 1 càng giống nghĩa), **không bị ảnh hưởng bởi độ dài (norm)**. **Euclidean** đo khoảng cách thẳng, **bị ảnh hưởng bởi scale/norm**. Với text embedding ta quan tâm ý nghĩa chứ không phải độ dài văn bản, nên Cosine là mặc định; Euclidean hợp với image/geo (đo khoảng cách tuyệt đối).

Module B

B1 — Xử lý văn bản (chăm: đủ 5 bước pipeline)

python

```
import re
from collections import Counter

def clean_and_count(text):
    text = re.sub(r"\s+", " ", text).strip()      # 1. chuẩn hóa khoảng trắng
    text = text.lower()                           # 2. về chữ thường
    text = re.sub(r"[^\w\s]", "", text)           # 3. bỏ ký tự đặc biệt/dấu c
    words = text.split()                          # 4. tokenize
    freq = Counter(words)                         # 5. đếm tần suất
    most_common = freq.most_common(1)[0]
    print("Từ xuất hiện nhiều nhất:", most_common)
    return dict(freq)

raw = "  Khách HÀNG rất hài lòng!! Sản phẩm tốt, sản phẩm bền... Giao hàng NHANH
print(clean_and_count(raw))
# Các từ xuất hiện 2 lần (đồng hạng cao nhất): "hàng", "sản", "phẩm"
```

- *Câu phụ:* Phải **redaction (che mờ) PII** — thay số điện thoại bằng `[REDACTED]` **trước khi** embed/đẩy lên Vector DB cloud, để tuân thủ GDPR/PDPA (theo slide Day 8 — Data Cleaning).

B2 — Công thức → Python

python

```
import numpy as np

# a) MSE thuần Python
def mse_pure(y_true, y_pred):
    n = len(y_true)
    return sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred)) / n

# b) MSE NumPy (vector hóa)
def mse_numpy(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    return np.mean((y_true - y_pred) ** 2)

# c) ReLU NumPy
def relu(x):
    return np.maximum(0, np.array(x))
```

B3 — Broadcasting

- a) `C.shape = (3, 4)`. Cơ chế: `(3, 1)` và `(4, )` → NumPy coi `(4, )` như `(1, 4)`, rồi **kéo giãn** chiều có kích thước 1: hàng `(3, 1)` lặp theo cột, `(1, 4)` lặp theo hàng → cùng `(3, 4)`.
- b) `(x + y).shape = (3, 3)`. `(3, )` → `(1, 3)`, cộng với `(3, 1)` → `(3, 3)`.

B4 — F1-Score

python

```
def f1_score(tp, fp, fn):
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall)
    return round(precision, 3), round(recall, 3), round(f1, 3)

print(f1_score(8, 5, 2)) # (0.615, 0.8, 0.696)
```

Với TP=8, FP=5, FN=2: **Precision** $\approx$ **0.615**, **Recall** = **0.8**, **F1** $\approx$ **0.696**.

Module C

C1 — Thiết kế hệ thống chấm bài (khung ý)

- a) **Công nghệ**: OCR để quét/đọc chữ từ ảnh bài làm (cả tự luận viết tay lẫn phiếu); CV (**Computer Vision**) để nhận diện ô tô đáp án trên phiếu MCQ (chấm khách quan, nhanh); LLM/NLP để chấm **tự luận** (so khớp ngữ nghĩa với rubric, đánh giá lập luận).
- b) **Pipeline**: Thu thập dữ liệu (ảnh bài + đáp án mẫu) → Tiền xử lý (khử nhiễu ảnh, chuẩn hóa, OCR, làm sạch text) → Trích xuất đặc trưng (embedding câu trả lời, các tiêu chí rubric) → Lựa chọn mô hình (CV cho MCQ, LLM cho tự luận) → Đánh giá hiệu năng (so với điểm giáo viên thật, đo Accuracy/F1, sai số trung bình).
- c) **RAG cho chấm tự luận**: Index hóa **đáp án mẫu/rubric** vào Vector DB → khi chấm, **retrieve** đúng tiêu chí liên quan → ép LLM **chấm dựa trên tài liệu lấy được** thay vì "kiến thức tự suy diễn". RAG tốt hơn LLM thuần vì: (1) **Grounding** — bám đáp án thật, giảm hallucinate; (2) **cập nhật** rubric chỉ cần đổi index, không train lại; (3) **truy vết** được căn cứ cho điểm (citation). **3 pha RAG**: Indexing → Retrieval → Generation. (*GraphRAG hữu ích nếu rubric có quan hệ phân cấp/đa bước.*)

C2 — Triển khai LLM

- a) **Cây quyết định**: Prompting nếu few-shot đã đạt ~80%+; **RAG** khi cần **kiến thức mới/cập nhật/tra cứu tài liệu**; **Fine-tune** khi volume rất lớn (>50k req/ngày), cần văn phong riêng và ROI dương. Tình huống pháp chế nội bộ, dữ liệu nhạy cảm → ưu tiên **RAG self-host** (kiến thức tra cứu + bảo mật on-prem); có thể **kết hợp LoRA** để chỉnh văn phong.
- b) **LoRA: đóng băng (freeze)** toàn bộ trọng số gốc W_0 , chỉ chèn thêm cập nhật hạng thấp $\Delta W = B \cdot A$ và **chỉ train** A, B (rất ít tham số, ~1%). A là ma trận $d \times r$, B là $r \times k$ với rank $r \ll \min(d, k)$ (thường $r \in \{8, 16, 32, 64\}$). Khi inference: merge $W = W_0 + B \cdot A \Rightarrow$ **không tăng latency**. **QLoRA** = lượng tử hóa base model xuống **4-bit (NF4)** rồi gắn LoRA adapter → giảm VRAM ~3×, chạy được trên GPU phổ thông (RTX 3090 24GB), chất lượng chỉ giảm ~2-5%.
- c) **Guardrails — Secure Deployment**:
 - **Input guardrails** (chặn *trước* khi tốn token / trước khi LLM trả lời): validation (độ dài, ngôn ngữ, format), **injection detection** ("ignore all previous instructions...", "you are now DAN"), topic filter, rate limit.
 - **Output guardrails** (kiểm tra *sau* khi LLM trả lời): content filter (toxicity, **PII redaction**), **grounding check** (output có bám context không, ép citation), format validation (chặn link/data bịa), queue cho human khi confidence thấp.
 - Chống **Prompt Injection** (cả direct & indirect qua tài liệu RAG), chống **data leakage** (lộ system prompt/API key), kiểm soát **harmful output**. Triết lý: **Defense in depth** = input + LLM + output rails.

C3 — Đánh giá AI

- **Phần 1:**
 - a) **Precision** = $\frac{TP}{TP + FP} = \frac{8}{8 + 5} = \frac{8}{13} \approx \mathbf{62\%}$. **Recall** = $\frac{TP}{TP + FN} = \frac{8}{8 + 2} = \frac{8}{10} = \mathbf{80\%}$.
 - b) App trẻ em → ưu tiên **Recall cao**: bỏ lọt nội dung xấu (FN) = trẻ thấy nội dung độc hại = **thảm họa**, tệ hơn nhiều so với gán nhầm vài video tốt (FP). **Rule of thumb**: bỏ lọt = **thảm họa** → cần recall bất kể user có thấy hay không. Ngược lại, **duyet vay ngân hàng**: báo nhầm (duyet sai) = mất tiền → ưu tiên **Precision**. → Không có đáp án chung, tùy "**sai kiểu nào tệ hơn cho user**".
- **Phần 2:**
 - a) **Context Recall** — Retriever có mang về đủ thông tin cần để trả lời trọn vẹn không. **Faithfulness** — câu trả lời có bám 100% vào tài liệu hay tự bịa (hallucinate). **Answer Relevance** — câu trả lời có đi thẳng vào câu hỏi hay vòng vo.
 - b) **Faithfulness thấp + Context Recall cao** → tài liệu đúng nhưng LLM bịa thêm → **lỗi ở Generation** → sửa prompt (ép grounding, ép citation, giảm temperature). **Context Recall thấp** → retriever không tìm đủ → sửa **Indexing/Retrieval** (tối ưu Vector DB, Hybrid Search, tăng Top-K).

Module D

D1 — Logic routing

1. $(0.95) \geq 0.9$, action thường → **agent tự động trả lời**.
2. **gửi email** thuộc nhóm hành động đặc biệt → **chuyển con người** (bất kể (0.98)).
3. (0.75) thuộc $[0.7, 0.9)$, action thường → **đưa hàng đợi human review (priority thường)**.
4. **xóa dữ liệu** thuộc nhóm đặc biệt → **chuyển con người** (và $(0.6 < 0.7)$ cũng đã low).

D2 — An toàn & Đạo đức

- a) **Air Canada**: AI tạo ra "ảo giác" (**Hallucination**) — bịa thông tin sai trông như thật; doanh nghiệp **chịu trách nhiệm pháp lý** cho output của AI. Bài học: phải **thiết kế cho cái sai** (failure mode), bắt buộc **Grounding** (chỉ trả lời dựa trên tài liệu thật + citation), và có cơ chế từ chối khi không chắc ("Nghệ thuật nói Tôi không biết").
- b) **Nguyên tắc** (nêu 3, ví dụ): **Fairness** (không thiên vị/phân biệt), **Transparency** (giải thích được quyết định), **Accountability** (có người chịu trách nhiệm), **Privacy** (bảo vệ dữ liệu cá nhân), **Safety** (không gây hại). **Framework**: EU AI Act, NIST AI RMF, UNESCO AI Ethics, VN AI Strategy (nêu ≥ 2).
- c) **Trigger HITL** (nêu 3): hành động **không thể hoàn tác** (gửi email/xóa data/publish), **quyết định high-stakes** (chuyển tiền, đổi policy), **confidence thấp** (< 0.7), edge case, **chủ đề nhạy cảm** (y tế/pháp lý/tài chính). HITL là **feature** vì kết hợp AI + phán đoán con người đúng lúc làm **tăng độ tin cậy** sản phẩm, chứ không phải dấu hiệu AI kém.

D3 — Bias & Bảo mật

- a) **Bias dữ liệu lịch sử (historical/data bias)** → mô hình học và **khuếch đại định kiến** sẵn có. Nguồn gốc: dữ liệu huấn luyện lệch (95% nam), không phải lỗi thuật toán.
- b) Giảm bias (nêu 2): cân bằng/tái lấy mẫu dữ liệu (re-balance dataset); **loại bỏ feature nhạy cảm** (giới tính) và proxy của nó; kiểm tra **fairness metrics** theo nhóm; human review các quyết định loại CV; audit định kỳ.
- c) **PII: mask/redaction** các trường nhạy cảm (tên, SĐT, địa chỉ) **trước khi** embed/đưa lên cloud; chỉ giữ thông tin cần cho đánh giá năng lực; kiểm soát quyền truy cập (ACL) và tuân thủ GDPR/PDPA.

Gợi ý trọng số ôn tập

Module	Điểm	Trọng tâm slide
A — Toán	20	Đại số tuyến tính cho AI: ma trận nghịch đảo, dot/cross, L1/L2, cosine
B — Code	25	Làm sạch text, công thức→Python/NumPy, broadcasting, F1
C — AI & Sản phẩm	35	System design + OCR/CV/LLM, RAG/GraphRAG, LoRA/QLoRA, Guardrails, RAGAS, Precision/Recall
D — Logic & Đạo đức	20	HITL routing, Hallucination/Grounding (Air Canada), governance frameworks, bias, PII

Chúc ôn thi tốt! 🍀